

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Interface graphique

- **GUI** Graphical User Interface.
- **IHM** Interface Homme Machine.
- Une interface graphique assure la communication entre le programme et l'utilisateur.
- L'interface est constituée :
 - d'une fenêtre principale et d'éventuellement de fenêtres secondaires
 - de fenêtres de dialogue
- Une fenêtre est composée de composants graphiques : zones d'édition, boutons, listes déroulantes, cases à cocher, menus.
- L'utilisateur interagit avec le logiciel grâce à une souris et/ou un touchpad et/ou le clavier, écran tactile etc . . .

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

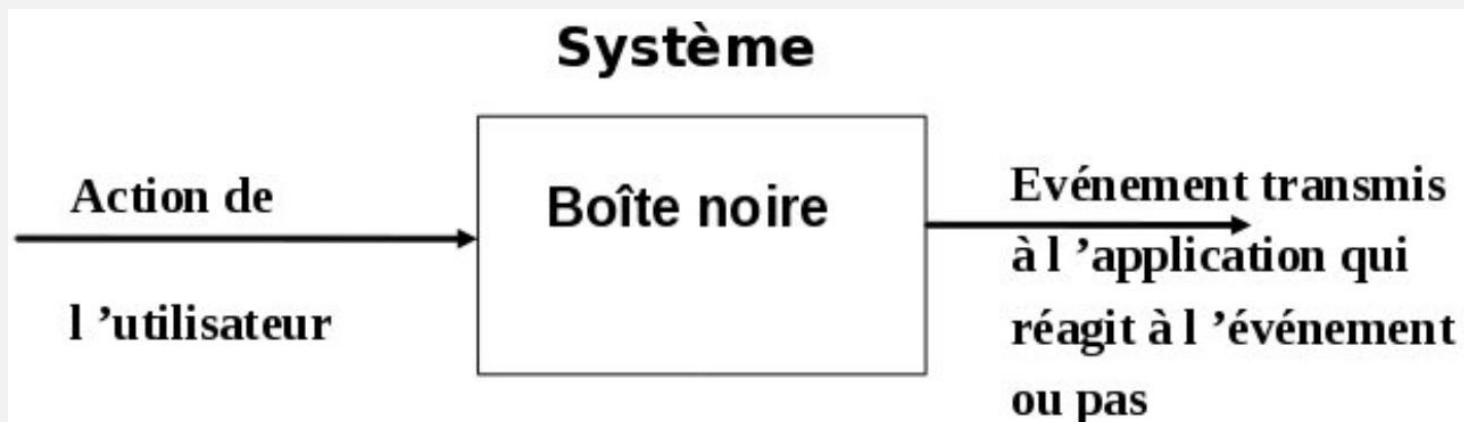
Programmation événementielle

- **Interactivité** : l'utilisateur agit sur l'interface graphique.
- A chaque action est associé un événement.
- A chaque événement est associé un morceau de code.
- Dès que vous lancez un programme avec une interface graphique :
 - une boucle infinie est générée.
 - le programme attend les actions de l'utilisateur (enfoncer une touche de clavier, clic souris etc . . .)
- Le programme traite l'action si elle a été prévue.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Programmation événementielle

- **Événement** : objet construit par le système en réponse à une action de l'utilisateur et qui contient toutes les informations concernant cette action.
 - **Clic souris** : les coordonnées de la souris au moment du clic, l'heure système.
 - **Clavier** : la touche utilisée, etc . . .



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

MVC

- La partie données qu'on appelle le **modèle**.
- La partie visualisation (Interface graphique) qu'on appelle **la vue**.
- La partie gestion des événements qu'on appelle le **contrôleur**.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les API d'interfaces en Java

- Java 1.1 (1996) : package **awt**(**A**bstr**a**ct **W**indow **T**oolkit).
- Java 1.2 : package **Swing** plus complet et light-weight.
- **JavaFX**: La plus récente.

AWT	Swing
Les composants AWT dépendent de la plate-forme.	Les composants Java swing ne dépendent pas de la plateforme. Ils sont purement scriptés en Java.
Lourd.	Légers, les composants sont conçus pour être au TOP des composants AWT et effectuer leurs tâches. Ces composants sont généralement légers car ils n'ont pas besoin d'objets OS natifs pour implémenter leurs fonctionnalités.
Les composants AWT en Java ne suivent pas l'architecture MVC (Model View Controller)	Les composants Swing en Java suivent le modèle MVC (Model View Controller).

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les API d'interfaces en Java

AWT	Swing
Les composants AWT sont moins nombreux que les composants Swing.	Les composants Java Swing sont bien plus nombreux.
Les composants AWT ont besoin du package java.awt .	Les composants Swing ont besoin du package javax.swing .
Les fonctionnalités avancées décrites par les composants Swing ne sont pas présentes dans les composants AWT.	Les composants Swing décrivent plusieurs fonctionnalités avancées telles que JTable , JtabbedPane , etc. Ces fonctionnalités avancées sont spécifiques uniquement aux composants Swing.
De nombreuses fonctions de composants AWT doivent être implémentées par le programmeur.	Les composants Swing offrent des fonctions intégrées.

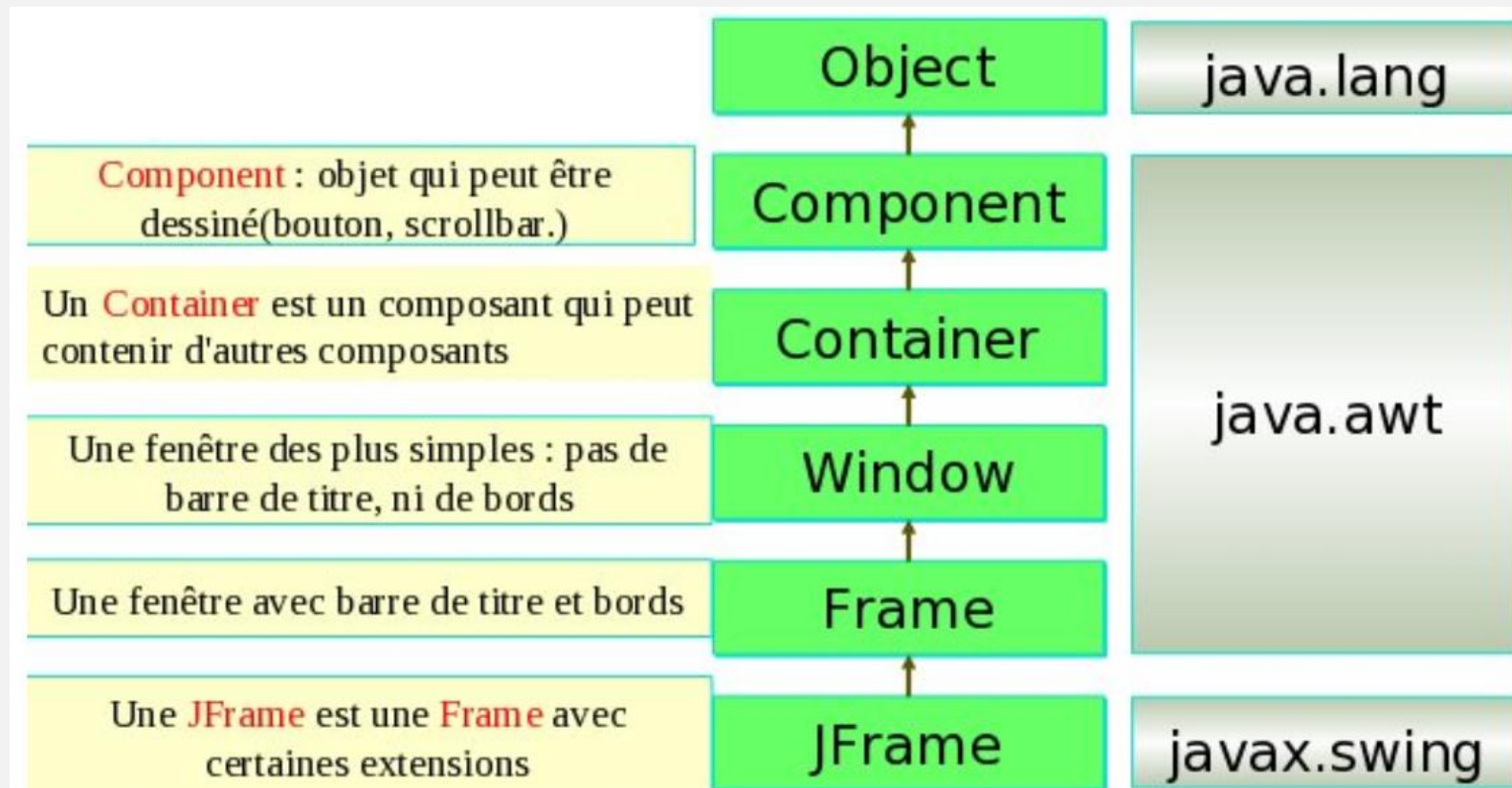
VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les fenêtres **JFrame**

- Pour une application graphique standard développée sur la machine locale, on utilise la classe **JFrame**.
- Pour une applet développée pour le web, c'est la classe **JApplet**.
- Dans la mesure du possible, éviter le mélange des composants swing et awt.
- Les classes **Swing** commencent par la lettre **J**

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les fenêtres JFrame



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

JComponent

- Tous les composants Swing qui commencent par J héritent de JComponent qui hérite de Container, sauf JFrame et JDialog qui héritent de Window.

```
▼ Object
  ▼ Component
    ▼ Container
      ▼ Window
        ▼ Frame
          ▼ JFrame
```

```
▼ Object
  ▼ Component
    ▼ Container
      ▼ Window
        ▼ Dialog
          ▼ JDialog
```

```
▼ Object
  ▼ Component
    ▼ Container
      ▼ JComponent
        ▼ JPanel
```

```
▼ Object
  ▼ Component
    ▼ Container
      ▼ JComponent
        ▼ JOptionPane
```

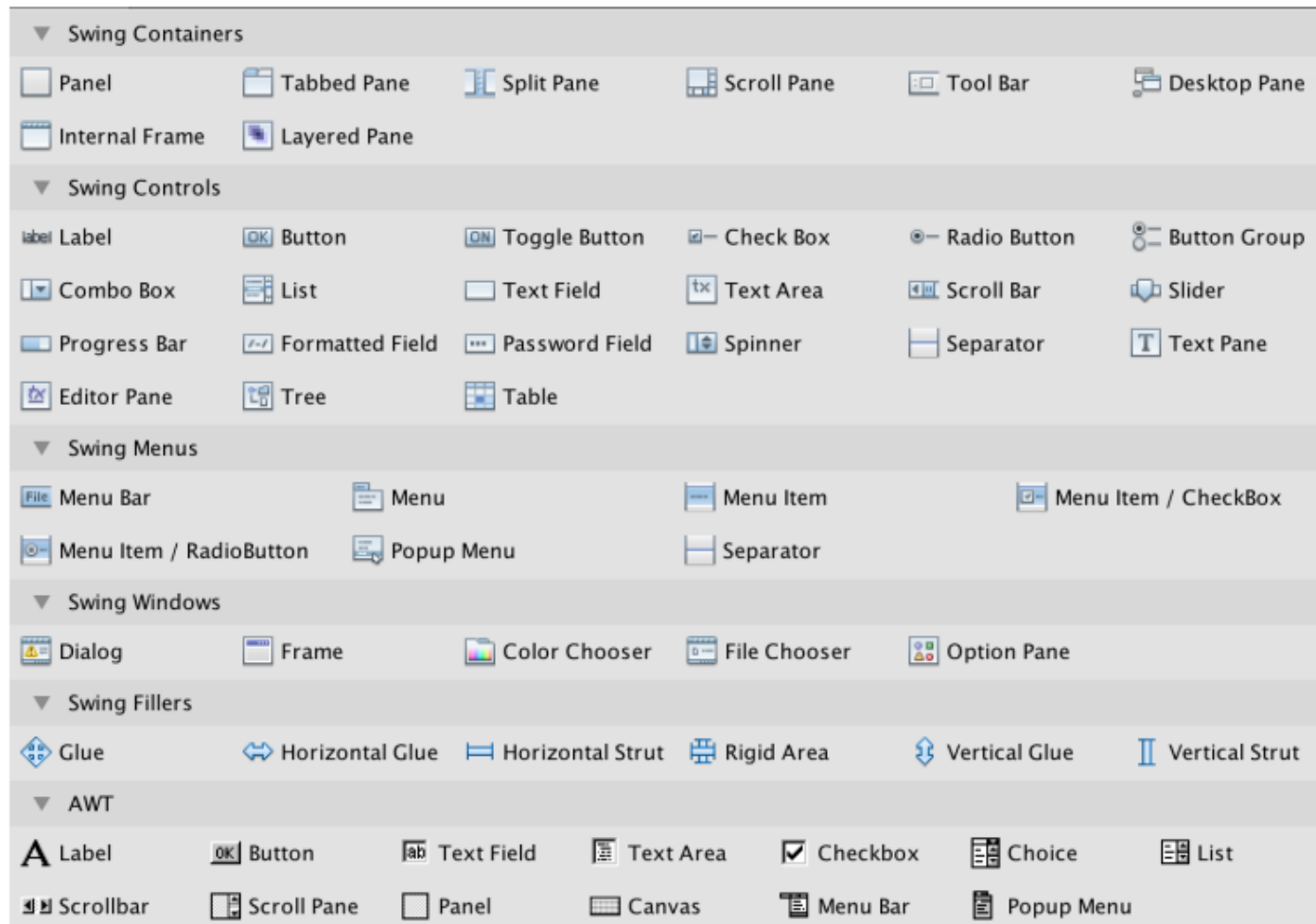
```
▼ Object
  ▼ Component
    ▼ Container
      ▼ JComponent
        ▼ AbstractButton
          ▼ JButton
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Composants graphiques

Les widgets

- JLabel
- JButton
- JToggleButton
- JCheckBox
- JRadioButton
- ButtonGroup
- JComboBox
- JList
- JTextField
- JTextArea
- JScrollBar
- JMenuBar
- JPopupMenu



Les containers

- JWindow
 - JFrame
 - JDialog
 - JFileDialog
- JPanel
 - Applet
- JTabbedPane
- JScrollPane

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Composants texte



JTextField

City:

JPasswordField

Enter the password:

JTextArea :

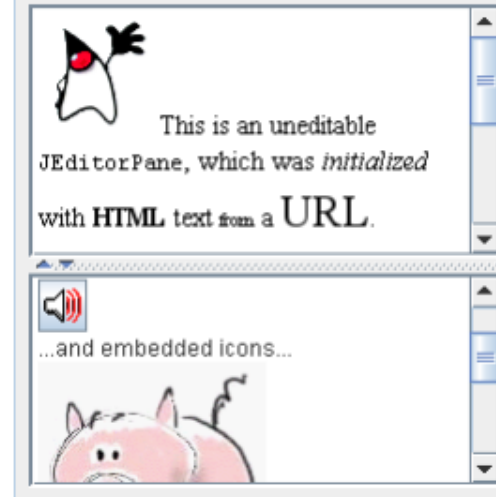
texte simple multilignes

Ascenseurs :

JScrollPane

This is an editable JTextArea. A text area is a "plain" text component, which means that although it can display text in any font, all of the text is in the same font.

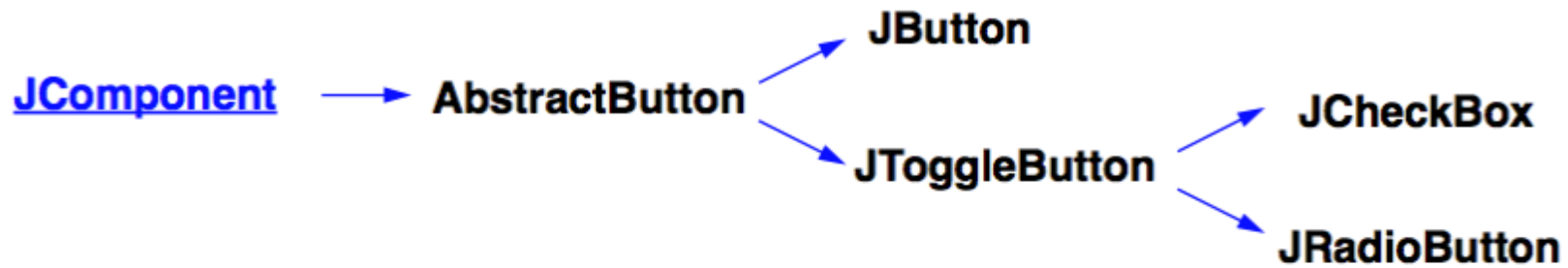
Styled Text



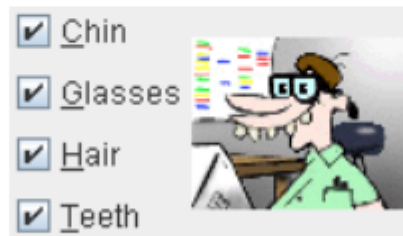
JEditorPane : texte avec styles compatible HTML et RTF

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Composants bouton



JButton



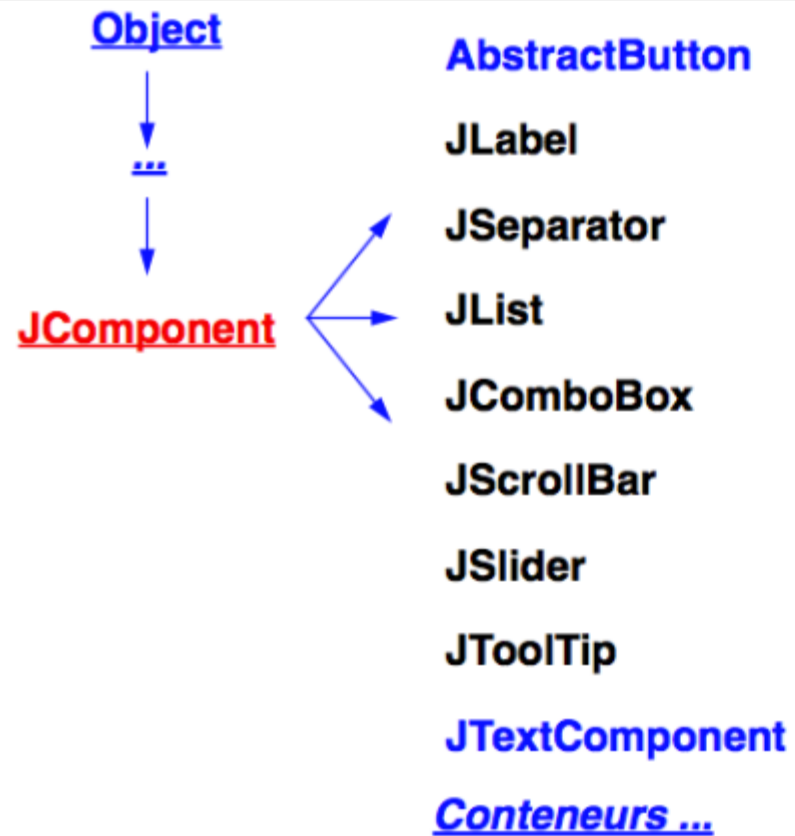
JCheckbox :
choix indépendants



JRadioButton :
choix exclusif : cf. **ButtonGroup**

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

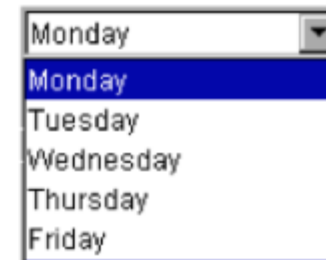
Composants interacteur



[JLabel](#)



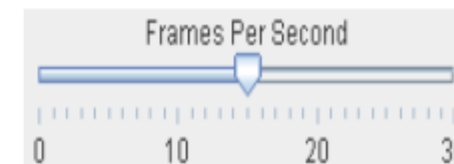
[JList](#)



[JComboBox](#)



[JScrollBar](#)

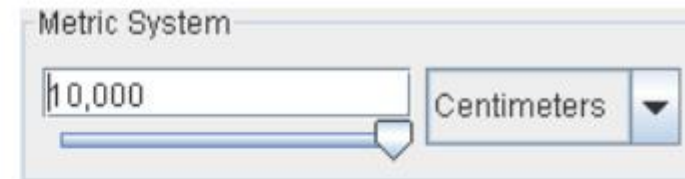
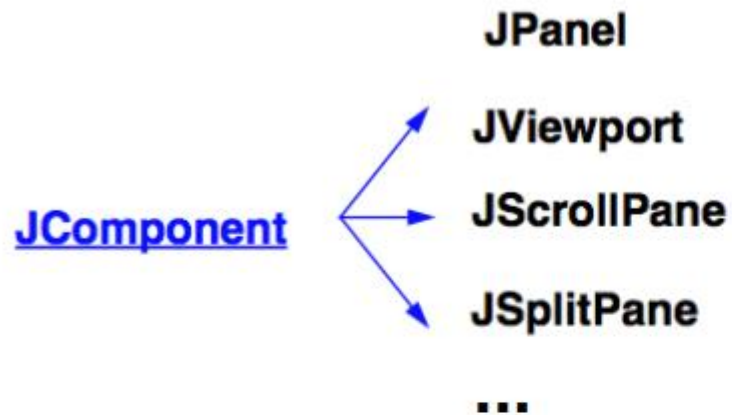


[JSlider](#)



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Conteneurs



JPanel: conteneur générique



JScrollPane:
avec ascenseurs intégrés



JSplitPane:
avec « diviseur » intégré

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

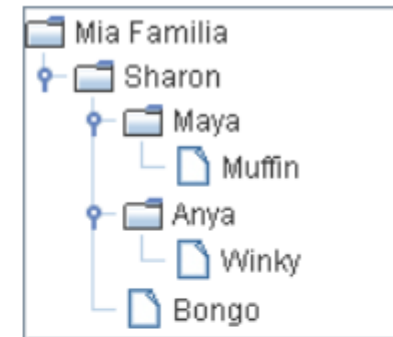
Conteneurs



JToolBar: barre d'outils
(sous la barre de menus)



JTabbedPane:
onglets



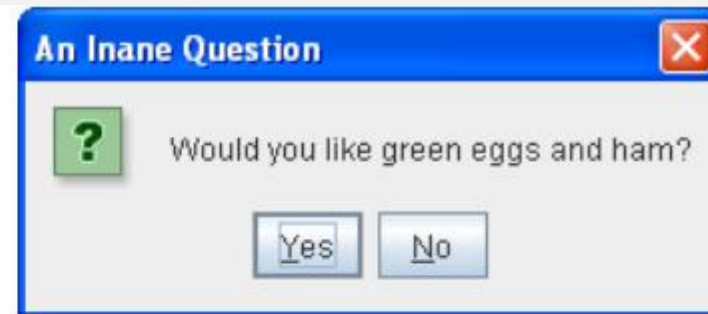
JTree

Host	User	Password	Last Modified
Biocca Games	Freddy	!#asf6Awwzb	Mar 16, 2006
zabble	ichabod	Tazb!34\$fZ	Mar 6, 2006
Sun Developer	fraz@hotmail.co...	AasW541!fbZ	Feb 22, 2006
Heirloom Seeds	shams@gmail....	bkz[ADF78!	Jul 29, 2005
Pacific Zoo Shop	seal@hotmail.c...	vbAf124%z	Feb 22, 2006

JTable

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Fenêtres



JFrame : fenêtre **principale** de l'application

JDialog : fenêtre **secondaire**

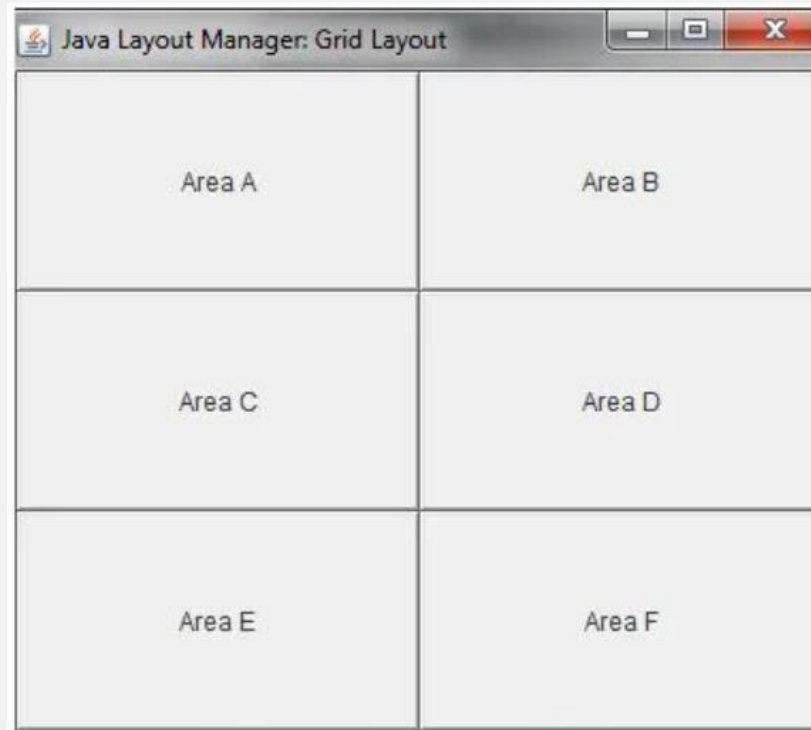
- dépendante de la **JFrame** (en théorie pas d'iconification séparée, toujours au dessus)

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les layout

- Les **LayoutManager** pour organiser votre fenêtre.
- **awt.CardLayout**
- **awt.FlowLayout**
- **awt.BorderLayout**
- **awt.GridLayout**
- **awt.GridBagLayout**

```
fj.setLayout(new FlowLayout(FlowLayout.LEFT));
```

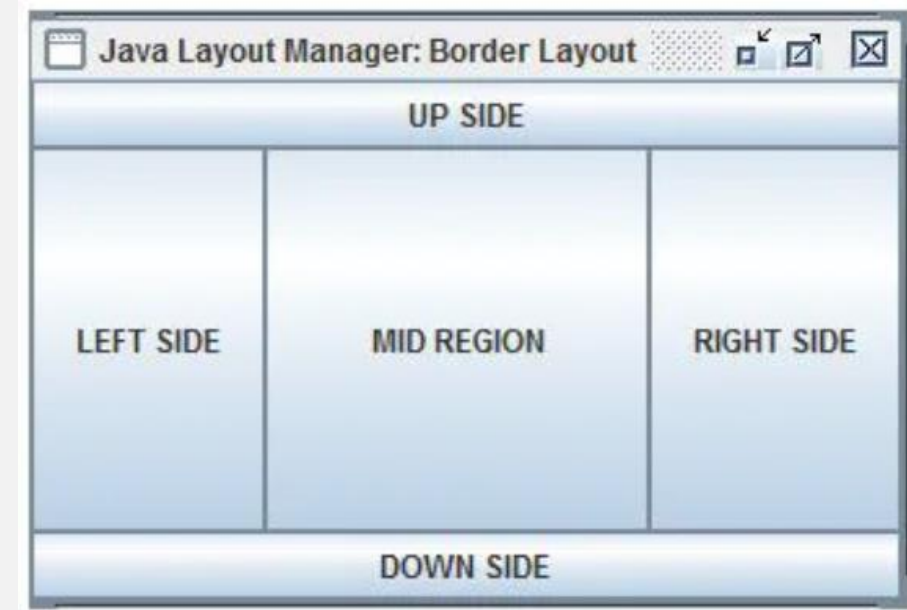


```
GridLayout gl = new GridLayout(3, 2);  
setLayout(gl);
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

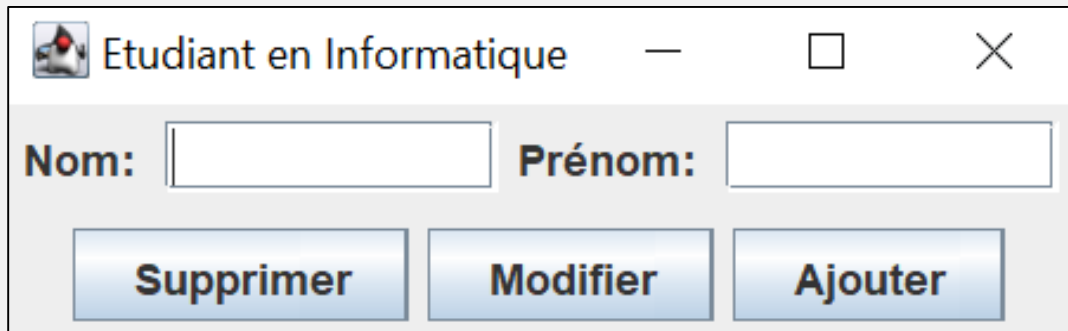
Les layout

```
pn1.setLayout(new BorderLayout());  
pn1.add(jbtn1, BorderLayout.NORTH);  
pn1.add(jbtn2, BorderLayout.SOUTH);  
pn1.add(jbtn3, BorderLayout.WEST);  
pn1.add(jbtn4, BorderLayout.EAST);  
pn1.add(jbtn5, BorderLayout.CENTER);
```



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exemple Création d'une fenêtre

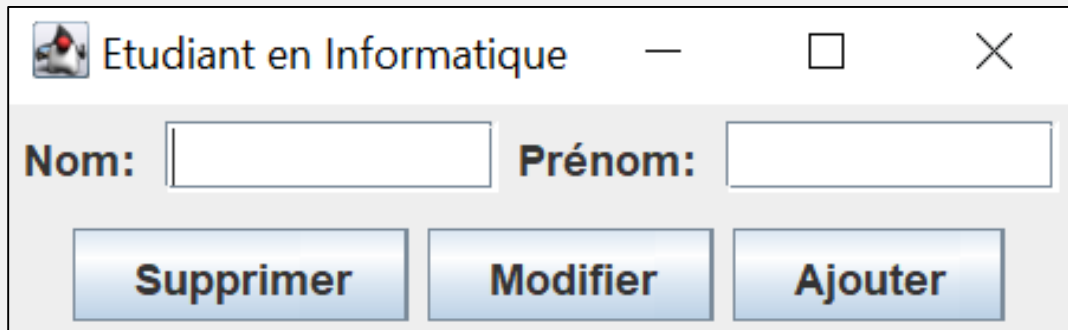


```
import java.awt.*;
import javax.swing.*;

public class FenetreEtudiant extends JFrame{
    // Les composants swing de la fenêtre
    private JLabel nomLabel, prenomLabel;
    private JTextField nomTextField,
        prenomTextField;
    private JButton supprimerButton,
        modifierButton, ajouterButton;
```

VIII. LES INTERFACES GRAPHIQUES

Exemple Création d'une fenêtre



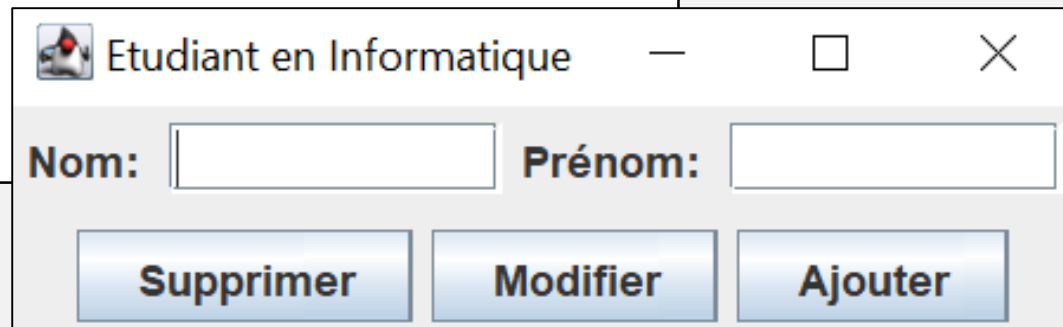
```
public void initComponents() {  
    nomLabel=new JLabel("Nom: ");  
    prenomLabel=new JLabel("Prénom: ");  
    nomTextField=new JTextField(10);  
    prenomTextField=new JTextField(10);  
    supprimerButton=new JButton("Supprimer");  
    modifierButton=new JButton("Modifier");  
    ajouterButton=new JButton("Ajouter");  
  
    JPanel nomPrenomPanel=new JPanel();  
    nomPrenomPanel.add(nomLabel);  
    nomPrenomPanel.add(nomTextField);  
    nomPrenomPanel.add(prenomLabel);  
    nomPrenomPanel.add(prenomTextField);  
  
    JPanel boutonsPanel=new JPanel();  
    boutonsPanel.add(supprimerButton);  
    boutonsPanel.add(modifierButton);  
    boutonsPanel.add(ajouterButton);  
  
    this.add(nomPrenomPanel, BorderLayout.NORTH);  
    this.add(boutonsPanel, BorderLayout.SOUTH);  
}  
}
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exemple Création d'une fenêtre

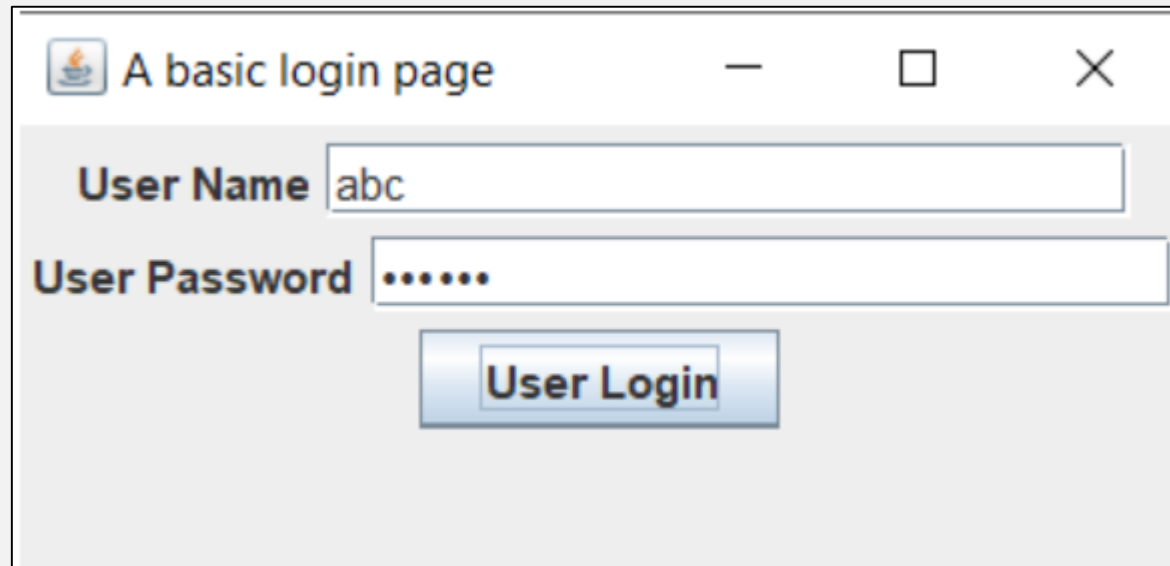
```
public class Test {  
    public static void main(String[] args) {  
        FenetreEtudiant f1=new FenetreEtudiant();  
    }  
}
```

```
public FenetreEtudiant() {  
    this initComponents();  
    this.setTitle("Etudiant en Informatique");  
    this.setSize(300,500);  
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    //Adapter la taille de la fenêtre à ses composants  
    this.pack();  
    this.setVisible(true);  
}
```

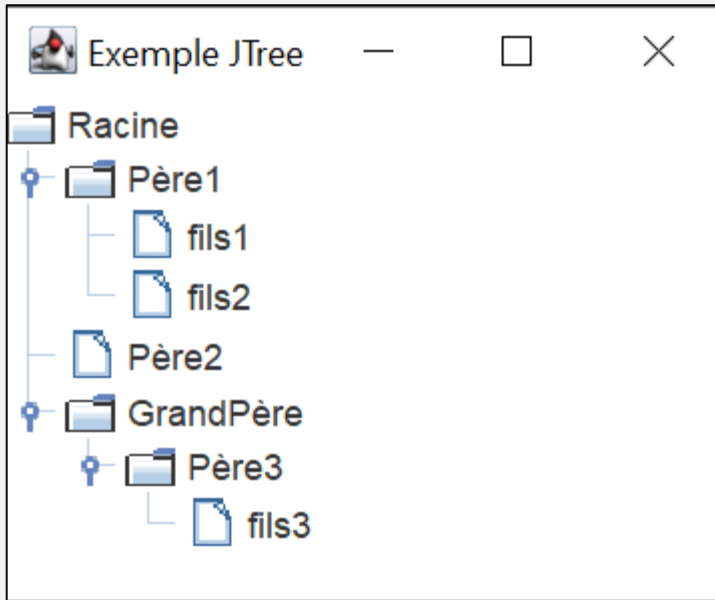


VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exercice



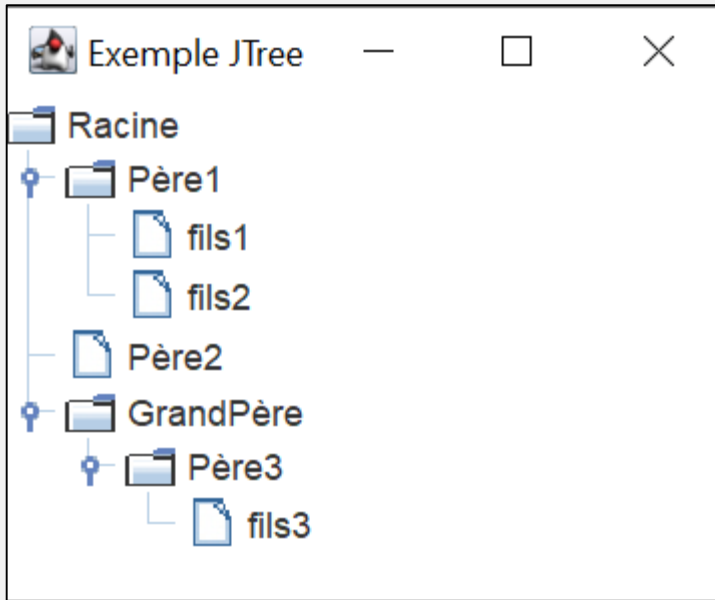
Exemple JTree



```
public void initComponents() {
    // Créer la racine
    DefaultMutableTreeNode racine=new
    DefaultMutableTreeNode("Racine");
    // Créer les noeuds fils
    DefaultMutableTreeNode p1=new
    DefaultMutableTreeNode("Père1");
    DefaultMutableTreeNode p2=new
    DefaultMutableTreeNode("Père2");
    DefaultMutableTreeNode gp=new
    DefaultMutableTreeNode("GrandPère");
    // Créer les noeuds fils
    DefaultMutableTreeNode f1=new
    DefaultMutableTreeNode("fils1");
    DefaultMutableTreeNode f2=new
    DefaultMutableTreeNode("fils2");
    DefaultMutableTreeNode f3=new
    DefaultMutableTreeNode("Père3");
    DefaultMutableTreeNode f4=new
    DefaultMutableTreeNode("fils3");
    DefaultMutableTreeNode f5=new
    DefaultMutableTreeNode("fils4");
}
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exemple JTree



```
public void initComponents() {  
    //...  
    // Ajouter les noeuds fils à leurs parents  
    racine.add(p1);  
    racine.add(p2);  
    racine.add(gp);  
    p1.add(f1);  
    p1.add(f2);  
    gp.add(f3);  
    f3.add(f4);  
    f3.add(f4);  
  
    tree=new JTree(racine);  
    this.add(tree);  
}
```

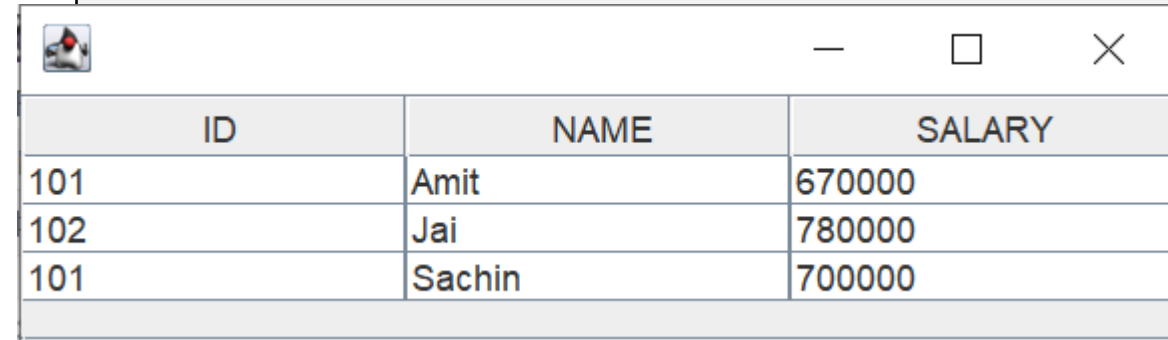
Example JTable

```
import javax.swing.*;
public class JTableExample extends JFrame{
    private JTable table;

    public JTableExample() {
        initComponents();
    }

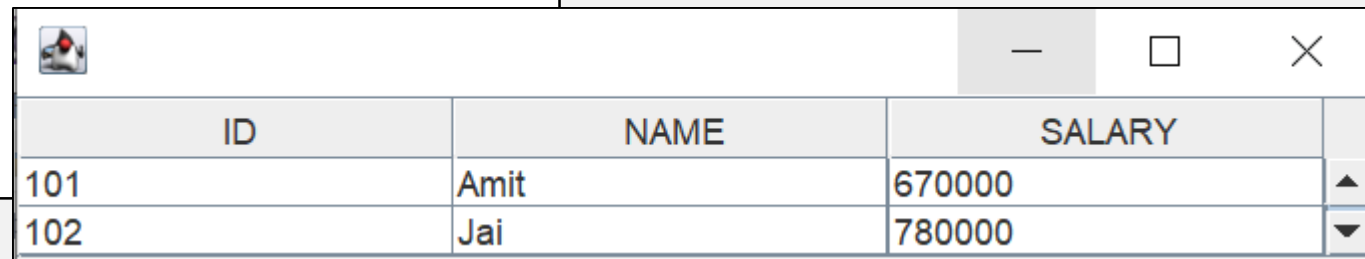
    public void initComponents() {
        String data[][]={ {"101","Amit","670000"},
                           {"102","Jai","780000"},
                           {"101","Sachin","700000"} };

        String column[]={"ID","NAME","SALARY"};
        table=new JTable(data,column);
        JScrollPane sp=new JScrollPane(table);
        this.add(sp);
    }
}
```



ID	NAME	SALARY
101	Amit	670000
102	Jai	780000
101	Sachin	700000

```
public class Test {
    public static void main(String[] args) {
        JTableExample f1=new JTableExample();
    }
}
```



ID	NAME	SALARY
101	Amit	670000
102	Jai	780000

```

import javax.swing.*;

public class JListExample extends JFrame{
    private JList<String> liste;

    public JListExample() {
        initComponents();
        this.setSize(100,200);
        this.setVisible(true);
    }
    public void initComponents() {
        String week[]= { "Monday","Tuesday","Wednesday",
            "Thursday","Friday","Saturday","Sunday"};
        liste=new JList<String>(week);
        JScrollPane sp=new JScrollPane(liste);
        this.add(sp);
    }
}

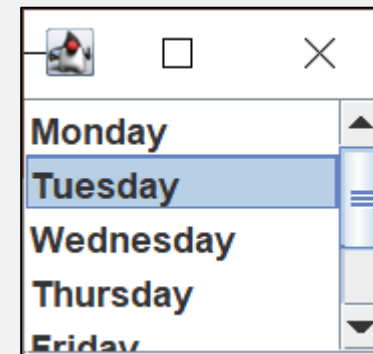
```

```

public class Test {
    public static void main(String[] args) {
        JListExample f1=new JListExample();
    }
}

```

Exemple JList<T>



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Rappel

2.1.2.1 La classe JButton

Elle permet de définir des boutons sur lesquels on peut cliquer. Ses principales méthodes sont :

JButton(String) construction avec définition du texte contenu dans le bouton

JButton(ImageIcon) construction avec une icône dans le bouton

JButton(String,ImageIcon) construction avec définition du texte et d'une icône dans le bouton

String getText() qui retourne le texte contenu dans le bouton

void setText(String) qui définit le texte contenu dans le bouton

void addActionListener(ActionListener) pour associer l'objet qui traitera les clics sur le bouton (voir 2.1.6.3)



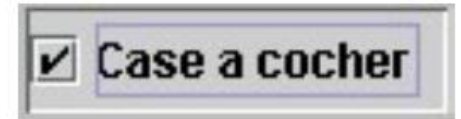
VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Rappel

2.1.2.2 La classe JCheckBox

Elle permet de définir des cases à cocher. Ses principales méthodes sont :

JCheckBox(String) construction avec définition du texte contenu dans la case à cocher



JCheckBox(String,boolean) construction avec en plus définition de l'état initial de la case à cocher

boolean isSelected() qui retourne l'état de la case à cocher (cochée ou non).

void setSelected(boolean) qui définit l'état de la case à cocher (cochée ou non).

String getText() qui retourne le texte contenu dans la case à cocher

void setText(String) qui définit le texte contenu dans la case à cocher

void addActionListener(ActionListener) pour associer l'objet qui traitera les actions sur la case à cocher (voir 2.1.6.3)

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Rappel

2.1.2.3 La classe JLabel

Elle permet de définir des textes fixes. Ses principales méthodes sont :

JLabel(String) qui construit le titre avec son contenu

void setText(String) qui définit le texte

String getText() qui retourne le texte



Ceci est un Titre !

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Rappel

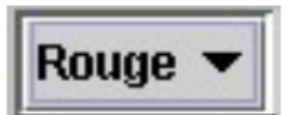
2.1.2.4 La classe JComboBox

Elle permet de définir des boîtes permettant de choisir une valeur parmi celles proposées. Ses principales méthodes sont :

JComboBox(Object[]) construction avec définition de la liste. On peut utiliser un tableau de chaînes de caractères (**String**) ou de toute autre classe d'objets.

void addItem(Object) qui ajoute une valeur possible de choix

int getSelectedIndex() qui retourne le numéro du choix actuel



Object getSelectedItem() qui retourne l'objet associé au choix actuel. Attention l'objet retourné est de classe **Object**, il faudra utiliser l'opérateur de coercition pour le transformer en sa classe d'origine (**String** par exemple).

void setSelectedIndex(int) qui sélectionne un élément défini par son numéro

void addActionListener(ActionListener) pour associer l'objet qui traitera les choix faits (voir 2.1.6.3)

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

2.1.2.5 La classe JList

Elle permet de définir des listes non déroulantes (pour disposer de listes avec ascenseur il faut faire appel à un contenant possédant des ascenseurs comme **JScrollPane**). La sélection dans ces listes peut porter sur 1 ou plusieurs objets. Ses principales méthodes sont :

JList(Object[]) construction avec définition de la liste. On peut utiliser un tableau de chaînes de caractères (**String**) ou de toute autre classe d'objets.

setListData(Object[]) définition de la liste. On peut utiliser un tableau de chaînes de caractères (**String**) ou de toute autre classe d'objets.

void setVisibleRowCount(int) qui définit le nombre d'éléments visibles sans ascenseur

int getSelectedIndex() qui retourne le numéro du premier élément sélectionné

int[] getSelectedIndices() qui retourne les numéros des éléments sélectionnés

Object getSelectedValue() qui retourne le premier objet sélectionné. Attention l'objet retourné est de classe **Object**, il faudra utiliser l'opérateur de coercition pour le transformer en sa classe d'origine (**String** par exemple).

Object[] getSelectedValues() qui retourne les objets actuellement sélectionnés.

void setSelectedIndex(int) qui sélectionne l'élément désigné par son numéro

void setSelectedIndices(int[]) qui sélectionne les éléments désignés par leurs numéros

void clearSelection() qui annule toutes les sélections

void addListSelectionListener(ListSelectionListener) pour associer l'objet qui traitera les sélections dans la liste (voir 2.1.6.3)



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

2.1.2.9 Les classes JTextField et JTextArea

Elles permettent de définir des zones de texte éditables sur une ou plusieurs lignes. Elles ont en commun un certain nombre de méthodes dont voici les principales :

void copy() qui copie dans le bloc-notes du système la partie de texte sélectionnée

void cut() qui fait comme **copy** puis supprime du texte la partie sélectionnée

void paste() qui fait copie dans le texte le contenu du bloc-notes du système

String getText() qui retourne le texte contenu dans la zone de texte

void setText(String) qui définit le texte contenu dans la zone de texte

int getCaretPosition() qui retourne la position du curseur d'insertion dans le texte (rang du caractère)

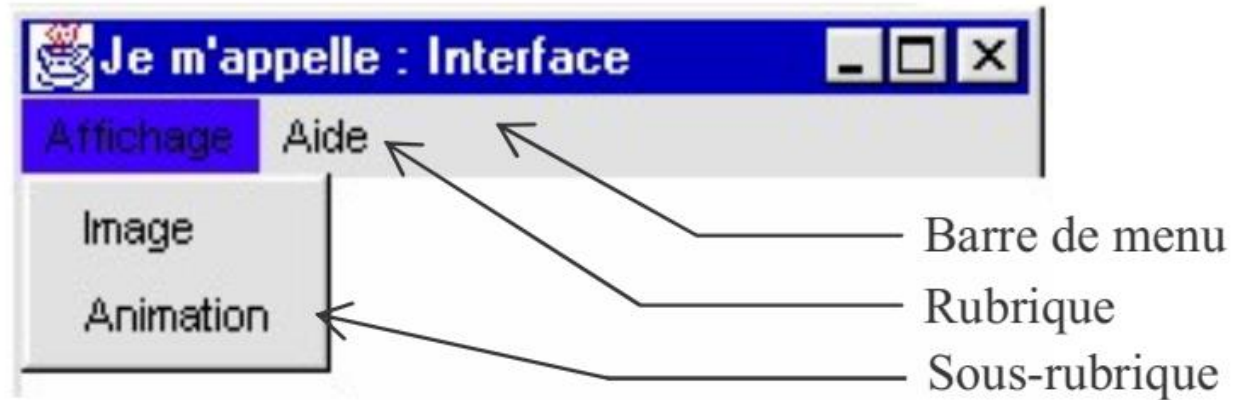
int setCaretPosition(int) qui place le curseur d'insertion dans le texte au rang indiqué (rang du caractère)

int moveCaretPosition(int) qui déplace le curseur d'insertion dans le texte en sélectionnant le texte depuis la position précédente.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

2.1.2.10 La classe Jmenubar

On peut doter une fenêtre d'une barre de menu qui s'affichera en haut et permettra d'accéder à des rubriques et sous rubriques. On utilisera pour cela les classes **JMenuBar**, **JMenu** et **JMenuItem**.



Remarque : la classe **Jmenu** hérite de **JMenuItem** de sorte que l'un des éléments d'une rubrique peut être lui-même une rubrique.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

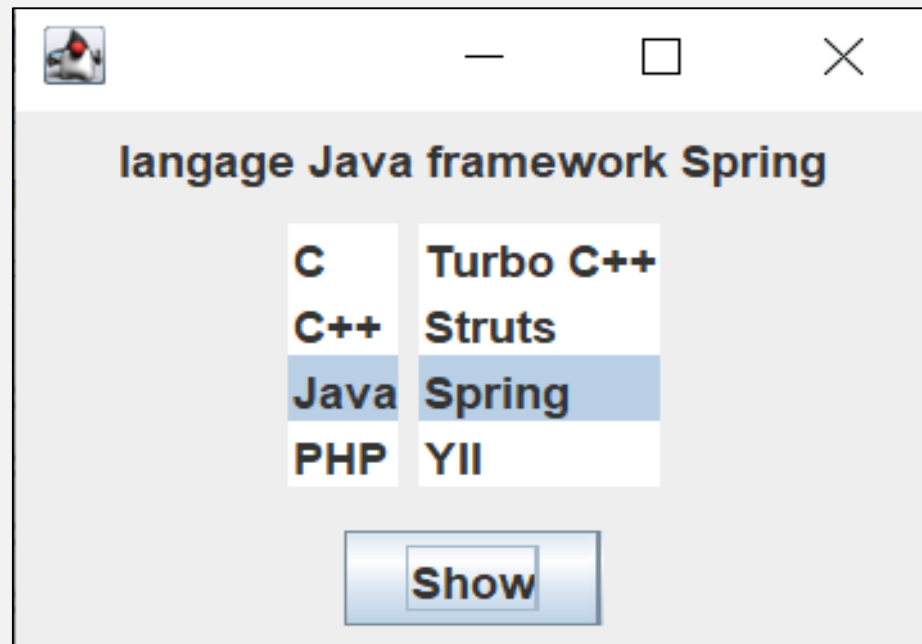
Remarques

- On peut insérer nos composants dans ***getContentPane()*** de la fenêtre courante.
- Eviter d'utiliser les méthodes ***setSize(Dimension)***, ***setLocation(Point)*** et ***setBounds(Rectangle)***. Au lieu de ça, on utilise des `LayoutManager` comme ça si on veut redimensionner la fenêtre, les composants sont automatiquement positionnés.
- On peut définir la taille préférée avec ***setPreferredSize(Dimension)***;
- La méthode ***setLocationRelativeTo()*** de la fenêtre permet de la positionner au centre de l'écran.
- ***setDefaultCloseOperation()*** permet d'indiquer le traitement en cas d'appuie sur le bouton fermer de la fenêtre. ***Jframe.EXIT_ON_CLOSE*** en paramètre permet de libérer toutes les ressources et sortir.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exercice

- Réaliser l'interface graphique suivante. Une fois je sélectionne un langage de programmation et un Framework, je modifie mon texte comme indiqué.



```
public class FenetreListe extends JFrame {  
    private JLabel langageFrameworkLabel;  
    private JButton showButton;  
    private JList<String> listLangages;  
    private JList<String> listFrameworks;  
  
    public void initComponents() {  
        langageFrameworkLabel = new JLabel();  
        showButton = new JButton("Show");  
        showButton.addActionListener(e->clickBListener());  
        Vector<String> langages=new Vector<String>();  
        langages.add("C");  
        langages.add("C++");  
        langages.add("Java");  
        langages.add("PHP");  
        listLangages = new JList<>(langages);  
        Vector<String> frameworks=new Vector<String>();  
        frameworks.add("Turbo C++");  
        frameworks.add("Struts");  
        frameworks.add("Spring");  
        frameworks.add("YII");  
        listFrameworks = new JList<>(frameworks);  
    }  
}
```

```

public void addComponents() {
    JPanel panel1=new JPanel();
    panel1.add(langageFrameworkLabel);
    JPanel panel2=new JPanel();
    panel2.add(listLangages);
    panel2.add(listFrameworks);
    JPanel panel3=new JPanel();
    panel3.add(showButton);
    this.add(panel1, BorderLayout.NORTH);
    this.add(panel2, BorderLayout.CENTER);
    this.add(panel3, BorderLayout.SOUTH);
}
FenetreListe() {
    initComponents();
    addComponents();
    this.setSize(500,500);
    this.setVisible(true);
    this.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE);
    this.pack();
}
public void clickBListener() {
    langageFrameworkLabel.setText("langage "+listLangages.getSelectedValue()+"
framework"+listFrameworks.getSelectedValue());}}

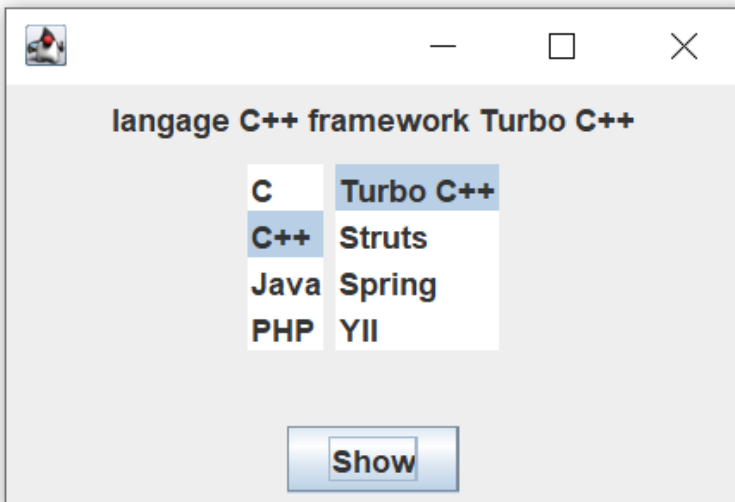
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

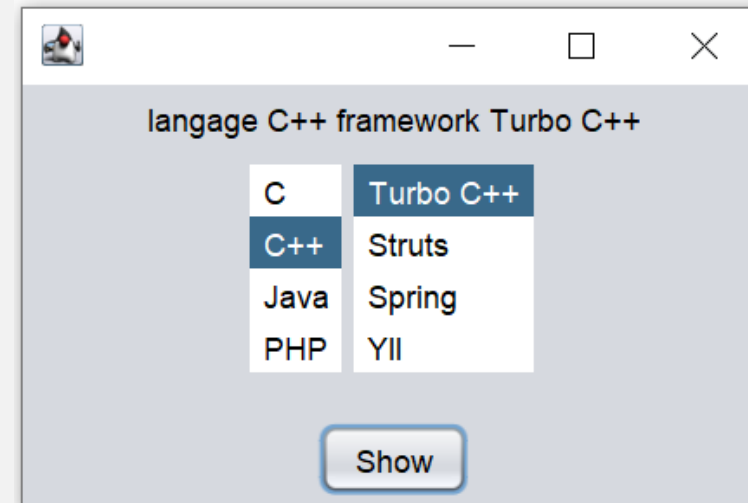
Look and Feel

Contrairement à awt, Swing prend en charge le **Look and Feel**.

```
import javax.swing.plaf.nimbus.NimbusLookAndFeel;
.....
public static void main(String args[]) throws UnsupportedLookAndFeelException {
    UIManager.setLookAndFeel(new NimbusLookAndFeel());
    // Le setLookAndFeel() doit venir avant la création de la fenêtre
    FenetreListe maFenetre=new FenetreListe();}
```



LookAndFeel(Metal): Par défaut



LookAndFeel(Nimbus)

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

JOptionPane

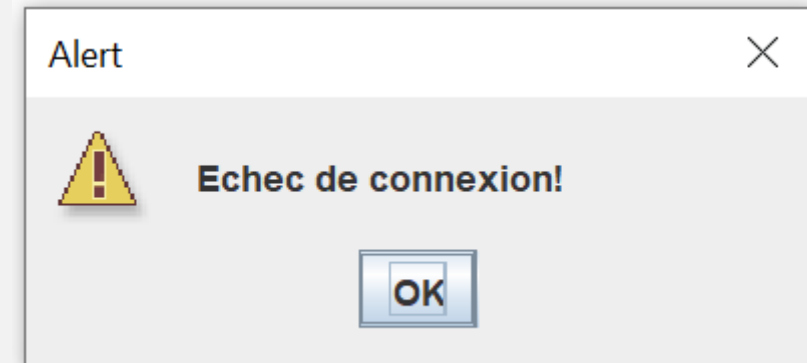
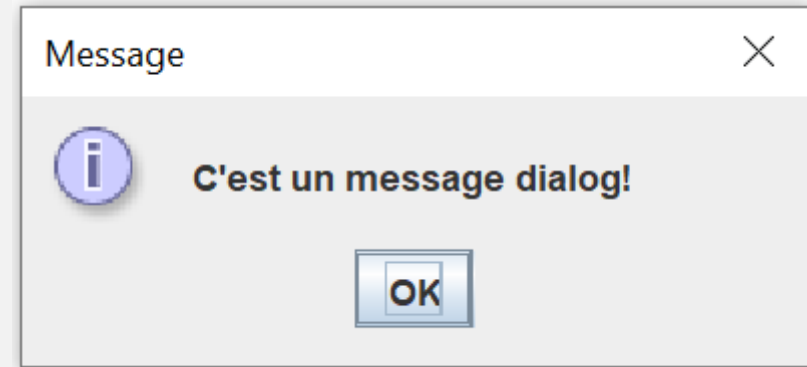
- La classe **JOptionPane** est utilisée pour fournir des boîtes de dialogue standard telles que la boîte de **message**, la boîte de **confirmation** et la boîte de **saisie**.
- Ces boîtes de dialogue sont utilisées pour afficher des informations ou obtenir des informations de l'utilisateur.
- La classe JOptionPane hérite de la classe JComponent.

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

JOptionPane

```
JOptionPane.showMessageDialog(  
    this,  
    "C'est un message dialog!"  
);
```

```
JOptionPane.showMessageDialog(  
    this,  
    "Echec de connexion!",  
    "Alert",  
    JOptionPane.WARNING_MESSAGE  
);
```

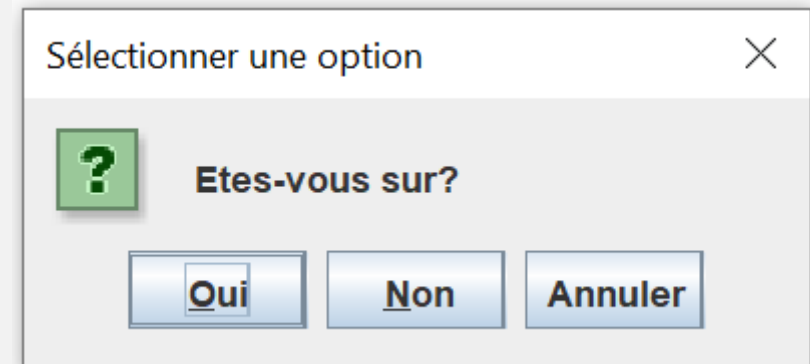
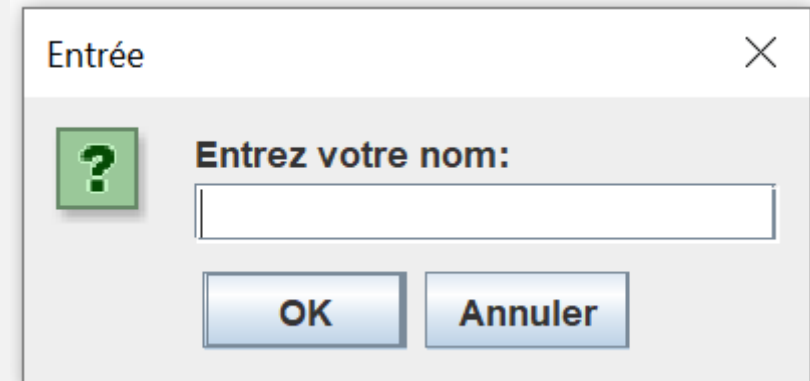


VIII. LES INTERFACES GRAPHIQUES AVEC SWING

JOptionPane

```
String nom = JOptionPane.showInputDialog(  
    this,  
    "Entrez votre nom:"  
);
```

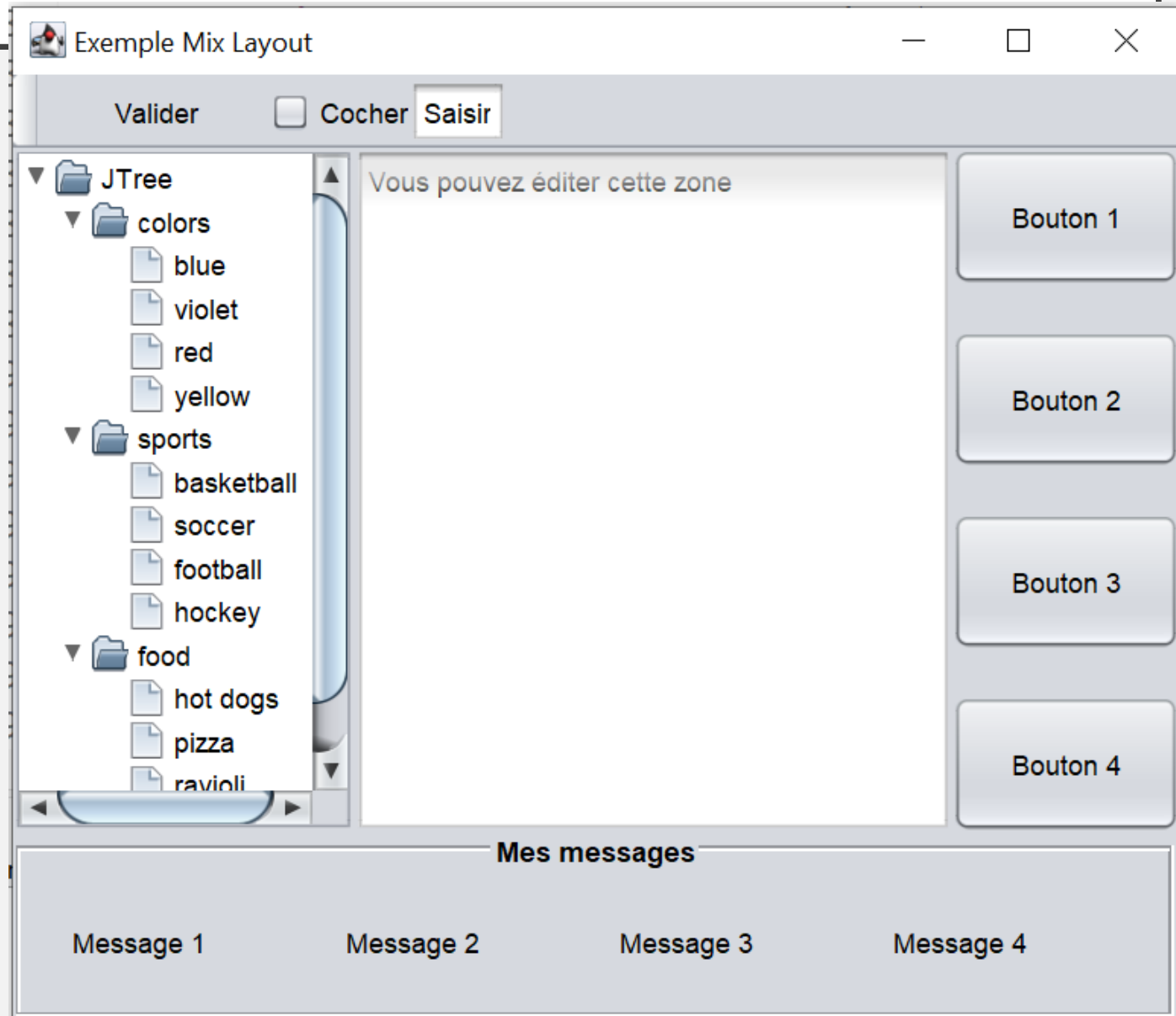
```
int res = JOptionPane.showConfirmDialog(  
    this,  
    "Etes-vous sur?"  
);
```



VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exercice

- Identifier les composants graphiques de l'interface suivante.
- Quels sont les différents layout pouvant être utilisés?
- Réaliser l'interface graphique suivante. Utiliser le thème Nimbus.



```
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.LayoutManager;

import javax.swing.*;
import javax.swing.border.TitledBorder;
import javax.swing.plaf.nimbus.NimbusLookAndFeel;

public class FenetreMixLayout extends JFrame{
    private JTree colorTree=new JTree();
    private JButton validerButton=new JButton("Valider");
    private JCheckBox cocherCheckox= new JCheckBox("Cocher",false);
    private JTextField saisirTextField =new JTextField("Saisir");
    private JTextArea editerTextArea =new JTextArea("Vous pouvez éditer cette zone");
    private JButton bouton1= new JButton("Bouton 1");
    private JButton bouton2= new JButton("Bouton 2");
    private JButton bouton3= new JButton("Bouton 3");
    private JButton bouton4= new JButton("Bouton 4");
    private JLabel message1= new JLabel("Message 1");
    private JLabel message2= new JLabel("Message 2");
    private JLabel message3= new JLabel("Message 3");
    private JLabel message4= new JLabel("Message 4");
```

```
private JToolBar creerToolBar() {  
    JToolBar result=new JToolBar();  
    result.add(validerButton);  
    result.add(cocherCheckox);  
    result.add(saisirTextField);  
    return result;  
}  
private JPanel creerBoutonsPanel() {  
    JPanel result=new JPanel();  
    result.setLayout(new GridLayout(4,1,20,20));  
    bouton1.setPreferredSize(new Dimension(100,20));  
    bouton2.setPreferredSize(new Dimension(100,20));  
    bouton3.setPreferredSize(new Dimension(100,20));  
    bouton4.setPreferredSize(new Dimension(100,20));  
    result.add(bouton1);  
    result.add(bouton2);  
    result.add(bouton3);  
    result.add(bouton4);  
    return result;  
}
```

```
private JPanel creerMessagesPanel() {  
    JPanel result=new JPanel();  
    LayoutManager lm=new FlowLayout(1,20,20);  
    result.setLayout(lm);  
    result.setBorder(BorderFactory.createTitledBorder(  
        BorderFactory.createEtchedBorder(), "Mes messages", TitledBorder.CENTER,  
        TitledBorder.TOP));  
    message1.setPreferredSize(new Dimension(100,20));  
    message2.setPreferredSize(new Dimension(100,20));  
    message3.setPreferredSize(new Dimension(100,20));  
    message4.setPreferredSize(new Dimension(100,20));  
    result.add(message1);  
    result.add(message2);  
    result.add(message3);  
    result.add(message4);  
    return result;  
}
```

```
private JScrollPane creerArbrePanel() {
    JScrollPane result=new JScrollPane(colorTree);
    result.setPreferredSize(new Dimension(150,300));
    return result;
}
public FenetreMixLayout(String title) {
    validerButton.setPreferredSize(new Dimension(100,20));

    this.add(creerToolBar(),BorderLayout.NORTH);
    this.add(creerMessagesPanel(),BorderLayout.SOUTH);
    this.add(creerBoutonsPanel(),BorderLayout.EAST);
    this.add(creerArbrePanel(),BorderLayout.WEST);
    this.add(editerTextArea,BorderLayout.CENTER);
    this.pack();
    this.setLocationRelativeTo(null);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setTitle(title);
    this.setVisible(true);

}
public static void main(String[] args) throws UnsupportedLookAndFeelException {
    UIManager.setLookAndFeel(new NimbusLookAndFeel());
    FenetreMixLayout maFenetre=new FenetreMixLayout("Exemple Mix Layout");
}}
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Traitement des événements

- Pour le traitement des événements, on peut:
 - Implémenter directement l'interface de contrôle (**ActionListener** par exemple qui contient la méthode `actionPerformed()`) ou hériter d'un modèle de contrôle (**MouseAdapter** par exemple).
 - Utiliser des classes internes privées.
 - Utiliser des classes anonymes.
 - Utiliser les expressions Lambda (**Depuis Java 8 et utilisé seulement pour ActionListener**).
 - Utiliser la référence sur méthode (**Depuis Java 8 et utilisé seulement pour ActionListener**).
- Pour associer un contrôleur à un événement on utilise la méthode **addxxxListener()**. On utilisera selon le cas ***addActionListener()***, ***addMouseListener()***, ***addKeyListener()***, etc,...

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Traitement des événements

- Les interfaces modèle **xxxListener**. Exemple: *ActionListener*, *MouseListener*, *KeyListener*,...
- Les classes modèle sont abstraites **xxxAdapter**. Exemple: *MouseAdapter*, *KeyAdapter*,...
- Remarque: *Il n'existe pas une classe ActionAdapter*, puisque l'interface **ActionListener** contient une seule méthode.
- Si on n'a pas besoin d'implémenter toutes les méthodes d'une interface, on utilise l'héritage des classes Adapter.
- **P.S.** Les classes de gestion des événements sont inclus dans *java.awt.event*.

MouseListener et MouseAdapter

```
public interface MouseListener extends EventListener {  
    public void mouseClicked(MouseEvent e);  
    public void mousePressed(MouseEvent e);  
    public void mouseReleased(MouseEvent e);  
    public void mouseEntered(MouseEvent e);  
    public void mouseExited(MouseEvent e);  
}
```

```
public abstract class MouseAdapter implements  
MouseListener, MouseWheelListener, MouseMotionListener {  
    protected MouseAdapter() {}  
    public void mouseClicked(MouseEvent e) {}  
    public void mousePressed(MouseEvent e) {}  
    public void mouseReleased(MouseEvent e) {}  
    public void mouseEntered(MouseEvent e) {}  
    public void mouseExited(MouseEvent e) {}  
    public void mouseWheelMoved(MouseWheelEvent e){}  
    public void mouseDragged(MouseEvent e){}  
    public void mouseMoved(MouseEvent e){}  
}
```

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les événements élémentaires

Type d'événement	événement	classe modèle interface modèle	classe d'événement	nom de méthode associée
Visibilité	devient visible	FocusAdapter FocusListener	FocusEvent	focusGained
	devient non visible			focusLost
Clavier	touche appuyée	KeyAdapter KeyListener	KeyEvent	keyPressed
	touche lâchée			keyReleased
	touche tapée			keyTyped
Souris	clic	MouseAdapter MouseListener	MouseEvent	mouseClicked
	le curseur rentre dans le composant			mouseEntered
	le curseur sort du composant			mouseExited
	un bouton de la souris appuyé			mousePressed
	un bouton de la souris lâché			mouseReleased
	souris déplacée avec un bouton appuyé	MouseMotionAdapter MouseMotionListener	MouseMotionEvent	mouseDragged
	souris déplacée			mouseMoved

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les événements des fenêtres

Fenêtre	événement	classe modèle interface modèle	classe d'événement	nom de méthode associée
normales : JFrame , JDialog et JApplet	La fenêtre devient active (elle recevra les saisies au clavier)	WindowAdapter WindowListener	WindowEvent	windowActivated
	La fenêtre devient inactive			windowDeactivated
	La fenêtre est fermée			windowClosed
	La fenêtre est en cours de fermeture (pas encore fermée)			windowClosing
	La fenêtre est mise ne icône			windowIconified
	La fenêtre est restaurée			windowDeiconified
	La fenêtre est visible pour la première fois			windowOpened

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Les événements des composants

- Ces éléments se produisent lorsqu'on coche une case, on sélectionne un élément dans une liste,..
- Chaque composants est susceptibles de réagir à certains de ces événements mais pas à tous.

Type d'événement	événement	classe d'événement
Action	action sur le composant (clic sur un bouton, choix dans un menu ...)	ActionEvent
Ajustement	déplacement d'un ascenseur	AdjustmentEvent
Changement	modification de la position d'un composant	ChangeEvent
Elément	modification de l'état d'un élément (liste, case à cocher ...)	ItemEvent
Document	modification dans un texte	DocumentEvent
Curseur	déplacement du curseur d'insertion dans un texte	CaretEvent
Sélection	sélection d'un élément dans une liste	ListSelectionEvent

Implémentation de ActionListener

```
public interface ActionListener extends EventListener {  
    public void actionPerformed(ActionEvent e);  
}
```

```
public class FenetreListe extends JFrame implements ActionListener{  
    private JLabel langageFrameworkLabel;  
    private JButton showButton;  
    private JList<String> listLangages;  
    private JList<String> listFrameworks;  
  
    public void initComponents() {  
        langageFrameworkLabel = new JLabel();  
        showButton = new JButton("Show");  
        showButton.addActionListener(this);  
        ...}  
    public void actionPerformed(ActionEvent e) {  
        langageFrameworkLabel.setText("langage  
        "+listLangages.getSelectedValue()+" framework  
        "+listFrameworks.getSelectedValue());  
    }  
    ...  
}
```

En cas de plusieurs actions sur les boutons, on peut utiliser dans actionPerformed() les conditions if(e.getSource==showButton)

Classes internes privées

```
public class FenetreListe extends JFrame {  
    .....{  
        showButton.addActionListener(new ActionAjouter());}  
        private class ActionAjouter implements ActionListener{  
            @Override  
            public void actionPerformed(ActionEvent e) {  
                langageFrameworkLabel.setText("langage "+listLangages.getSelectedValue()+"  
                framework "+listFrameworks.getSelectedValue());  
            }  
        }  
    }  
}
```

**La classe interne doit implémenter
ActionListener**

Classes anonymes

```
public class FenetreListe extends JFrame {
    private JLabel langageFrameworkLabel;
    private JButton showButton;
    private JList<String> listLangages;
    private JList<String> listFrameworks;

    public void initComponents() {
        langageFrameworkLabel = new JLabel();
        showButton = new JButton("Show");
        showButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                langageFrameworkLabel.setText("langage
"+listLangages.getSelectedValue()+" framework
"+listFrameworks.getSelectedValue());
            }
        });
        ...
    }
    ...
}
```

La classe anonyme est en fait une classe qui implémente ActionListener()

Expressions Lambda

```
showButton.addActionListener(e->clickBListener());
```

```
public void clickBListener() {  
    langageFrameworkLabel.setText("langage  
    "+listLangages.getSelectedValue()+"  
    framework"+listFrameworks.getSelectedValue());}
```

Référence sur méthode

```
showButton.addActionListener(this::clickBListener);
```

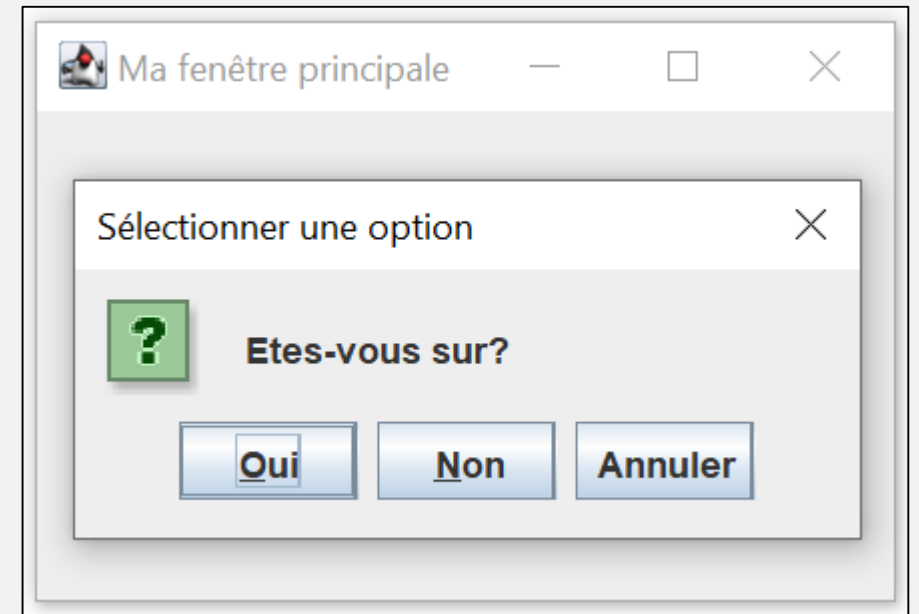
```
public void clickBListener(ActionEvent e) {  
    langageFrameworkLabel.setText("langage  
    "+listLangages.getSelectedValue()+" framework  
    "+listFrameworks.getSelectedValue());  
}
```

**Le paramètre ActionEvent e est obligatoire dans
méthode clickBListener(ActionEvent e) si on
veut utiliser la référence sur méthode**

VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exercice

- Soit une fenêtre simple.
- Si je clique sur la fermeture de la fenêtre, une fenêtre dialogue de confirmation s'affiche.
- Si je clique sur oui la fenêtre est fermée.
- Sinon la fenêtre n'est pas fermée.
- Quelles sont les interfaces d'événements que je peux implémenter?
- Quelles sont les classes d'événements que je peux étendre?
- Quelles sont les méthodes d'événements que je dois utiliser?



```
import java.awt.Dimension;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.JFrame;
import javax.swing.JOptionPane;

public class MyOptionPane extends JFrame {
    public MyOptionPane() {
        this.setSize(new Dimension(300, 200));
        this.setVisible(true);
        this.setTitle("Ma fenêtre principale");
        this.setLocationRelativeTo(null);
        this.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

        this.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                int res = JOptionPane.showConfirmDialog(MyOptionPane.this, "Etes-vous sur?");
                if (res == JOptionPane.YES_OPTION)
                    MyOptionPane.this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            }
        });
    }

    public static void main(String[] args) {
        new MyOptionPane();
    }
}
```

Exemple MVC

Exemple MVC

Prénom:

Nom:

Info

Nom ajouté avec succès : Karimi

Exemple MVC

Prénom:

Nom:

Info

Prénom ajouté avec succès : Karim

Exemple MVC

Prénom:

Nom:

Info

Bonjour Karim Karimi

Exemple MVC

Prénom:

Nom:

Info

Au revoir Karim Karimi

Le modèle

Exemple MVC

- >  JRE System Library [JavaSE-17]
- ▼  src
 - ▼  controller
 - >  Controller.java
 - ▼  model
 - >  Etudiant.java
 - >  test
 - ▼  view
 - >  FenetrePrincipale.java

```
public class Etudiant {  
    private String prenom;  
    private String nom;  
    public Etudiant(String prenom, String nom) {  
        super();  
        this.prenom = prenom;  
        this.nom = nom;  
    }  
    public String getPrenom() {  
        return prenom;  
    }  
    public void setPrenom(String prenom) {  
        this.prenom = prenom;  
    }  
    public String getNom() {  
        return nom;  
    }  
    public void setNom(String nom) {  
        this.nom = nom;  
    }  
}
```

```
public class FenetrePrincipale extends JFrame {
    private JLabel prenomLabel;
    private JLabel nomLabel;
    private JTextField prenomTextfield;
    private JTextField nomTextfield;
    private JButton ajouterPrenomButton;
    private JButton ajouterNomButton;
    private JButton direBonjourButton;
    private JButton direAuRevoirButton;
    public FenetrePrincipale(String title) {
        initComponents();
        this.pack();
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setVisible(true);
        this.setTitle(title);
    }
    public JLabel getPrenomLabel() {
        return prenomLabel;
    }
    public void setPrenomLabel(JLabel prenomLabel) {
        this.prenomLabel = prenomLabel;
    }
    //... Il faut ajouter tous les setters et getters
}
```

La vue

```
public class FenetrePrincipale extends JFrame {  
    public void initComponents() {  
        prenomLabel = new JLabel("Prénom: "); nomLabel = new JLabel("Nom: ");  
        prenomTextfield = new JTextField(12); nomTextfield = new JTextField(12);  
        ajouterPrenomButton = new JButton("Ajouter prénom");  
        ajouterNomButton = new JButton("Ajouter nom");  
        direBonjourButton = new JButton("Dire bonjour!");  
        direAuRevoirButton = new JButton("Dire au revoir!");  
        JPanel first = new JPanel();  
        GridLayout gL = new GridLayout(2,4);  
        gL.setHgap(25);  
        gL.setVgap(25);  
        first.setLayout(gL);  
        first.add(prenomLabel);  
        first.add(prenomTextfield);  
        first.add(ajouterPrenomButton);  
        first.add(direBonjourButton);  
        first.add(nomLabel);  
        first.add(nomTextfield);  
        first.add(ajouterNomButton);  
        first.add(direAuRevoirButton);  
        this.add(first);  
    }  
}
```

La vue

Le contrôleur

```
public class Controller {
    private Etudiant model;
    private FenetrePrincipale view;

    public Controller(Etudiant model, FenetrePrincipale view) {
        this.model = model;
        this.view = view;
        initView();
        initController();
    }

    public void initView() {
        view.getPrenomTextfield().setText(model.getPrenom());
        view.getNomTextfield().setText(model.getNom());
    }

    public void initController() {
        view.getAjouterPrenomButton().addActionListener(e -> ajouterPrenom());
        view.getAjouterNomButton().addActionListener(e -> ajouterNom());
        view.getDireBonjourButton().addActionListener(e -> direBonjour());
        view.getDireAuRevoirButton().addActionListener(e -> direAuRevoir());
    }
}
```

Le contrôleur

```
public class Controller {

    private void ajouterPrenom() {
        model.setPrenom(view.getPrenomTextField().getText());
        JOptionPane.showMessageDialog(null, "Prénom ajouté avec succès : " +
            model.getPrenom(), "Info", JOptionPane.INFORMATION_MESSAGE);
    }

    private void ajouterNom() {
        model.setNom(view.getNomTextField().getText());
        JOptionPane.showMessageDialog(null, "Nom ajouté avec succès : " + model.getNom(),
            "Info", JOptionPane.INFORMATION_MESSAGE);
    }

    private void direAuRevoir() {
        JOptionPane.showMessageDialog(null, "Au revoir " + model.getPrenom() + " " +
            model.getNom(), "Info", JOptionPane.INFORMATION_MESSAGE);
        System.exit(0);
    }

    private void direBonjour() {
        JOptionPane.showMessageDialog(null, "Bonjour " + model.getPrenom() + " " +
            model.getNom(), "Info", JOptionPane.INFORMATION_MESSAGE);
    }
}
```

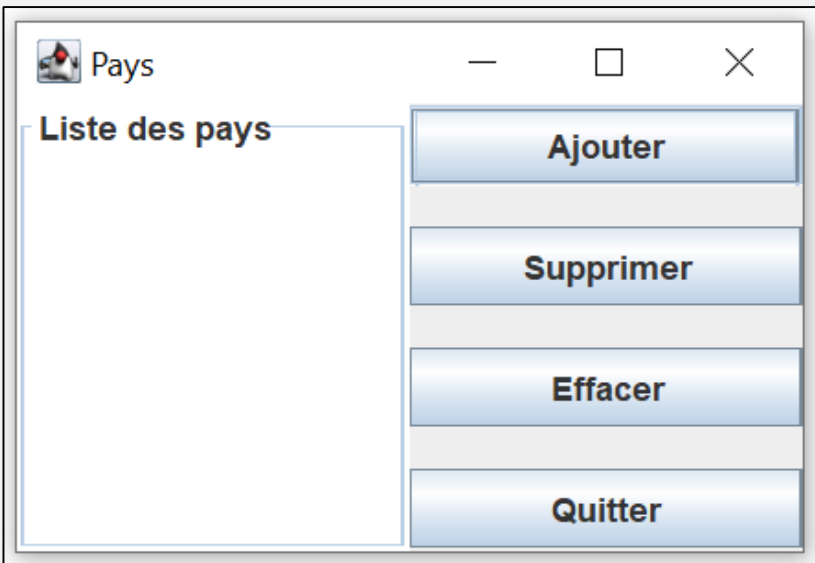
La classe principale

```
public class Test {  
  
    public static void main(String[] args) {  
        // Assemble all the pieces of the MVC  
        Etudiant e = new Etudiant("Karim", "Karimi");  
        FenetrePrincipale fP = new FenetrePrincipale("Exemple MVC");  
        Controller c = new Controller(e, fP);  
    }  
}
```

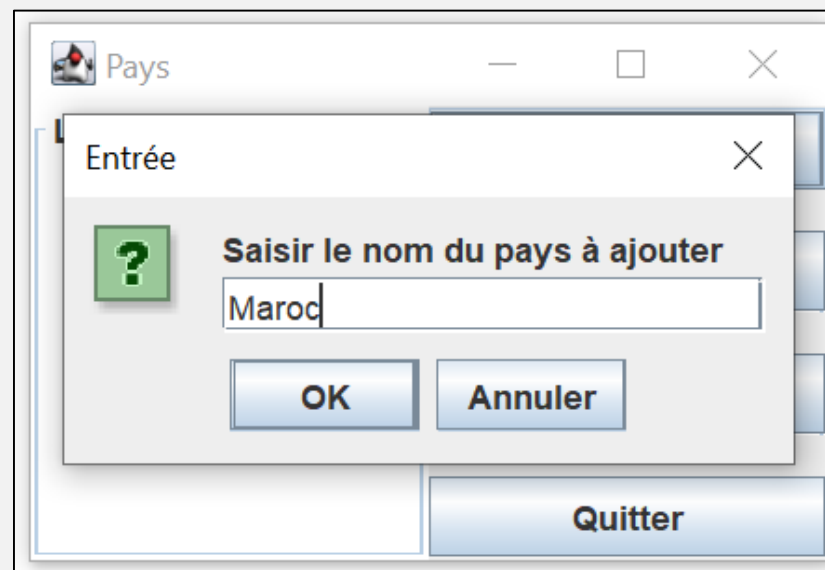
VIII. LES INTERFACES GRAPHIQUES AVEC SWING

Exercice JList et DefaultListModel

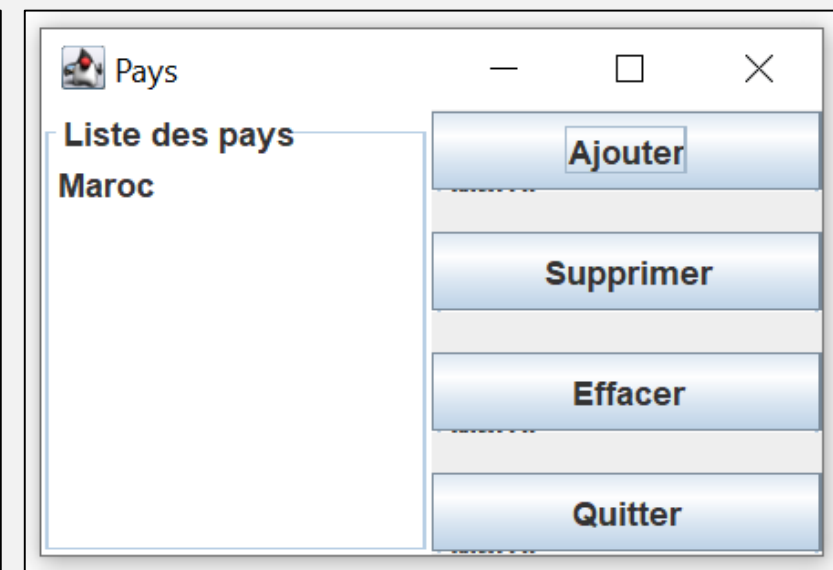
- Réaliser l'application suivante en utilisant le DefaultListModel (représentant le modèle) de la JList(représentant la vue). Implémenter ActionListener pour les événements.



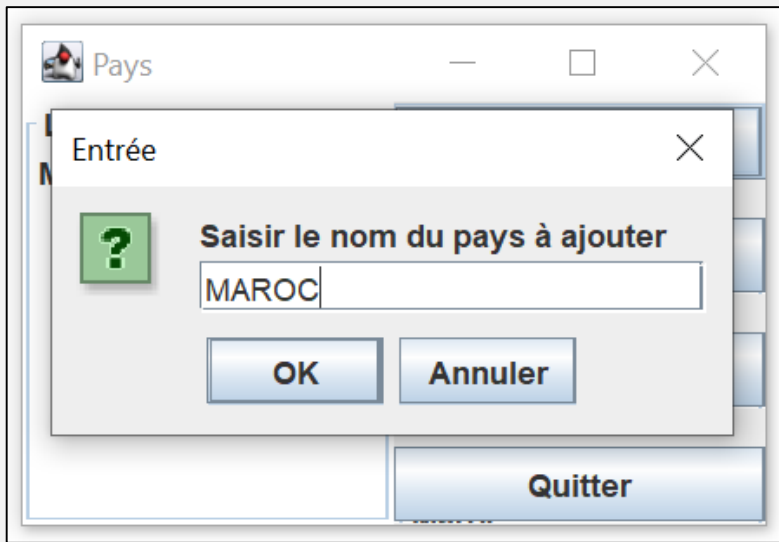
Fenêtre principale



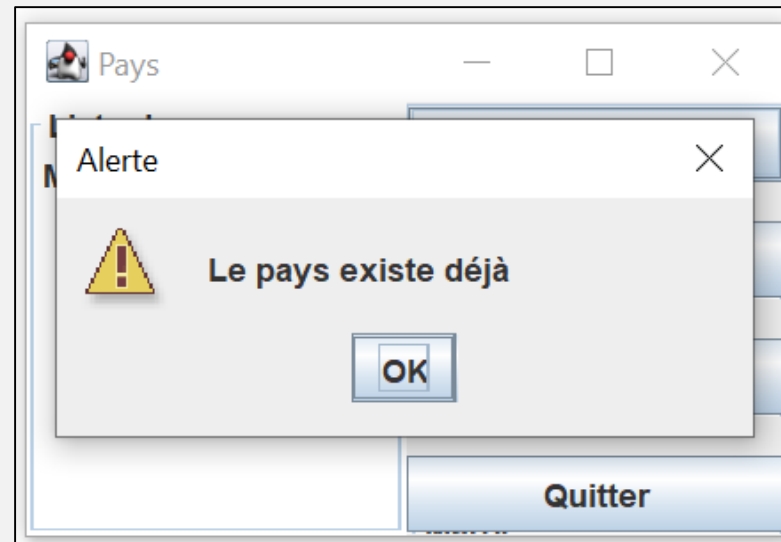
Clic sur le bouton ajouter



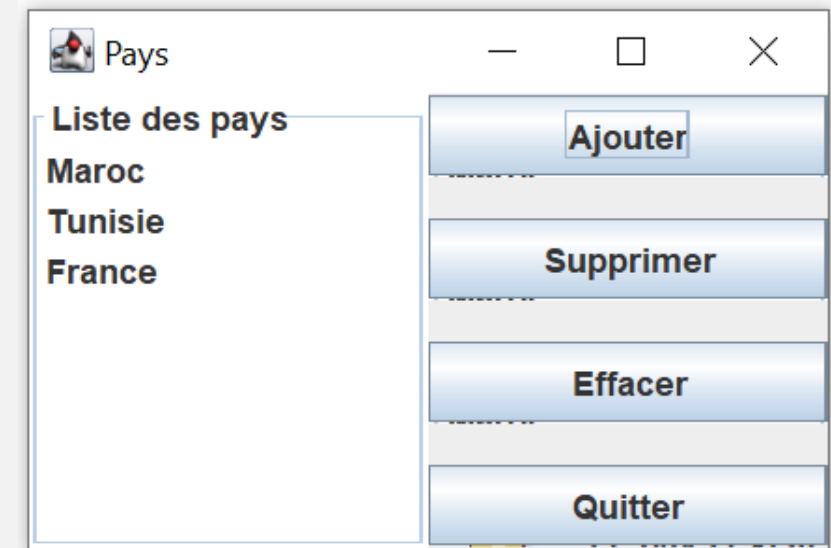
Le pays est ajouté à la liste



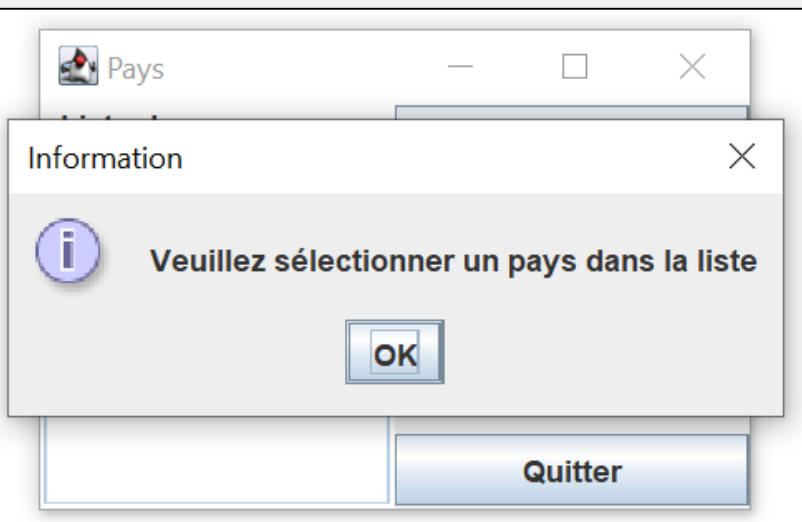
Essayer d'ajouter un pays qui existe déjà



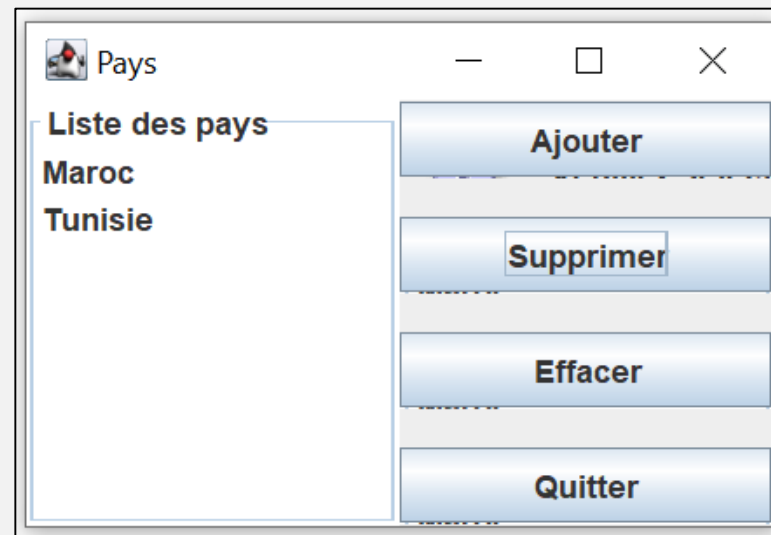
Pas possible d'ajouter un pays qui existe déjà



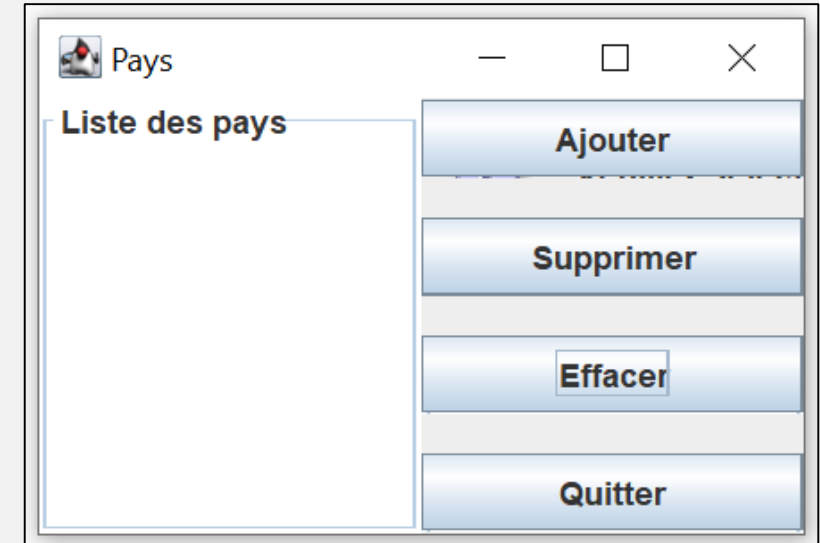
Ajouter plusieurs pays



Essayer de supprimer un pays non sélectionné



Supprimer le pays « France »



Effacer la liste

```
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.*;

public class MaFenetre extends JFrame implements ActionListener{
    JButton ajouterButton=new JButton("Ajouter");
    JButton supprimerButton=new JButton("Supprimer");
    JButton effacerButton=new JButton("Effacer");
    JButton quitterButton=new JButton("Quitter");
    DefaultListModel<String> paysListModel=new DefaultListModel<String>();
    JList<String> paysJListe = new JList<String>();

    public MaFenetre() {
        addComponents();
        pack();
        setTitle("Pays");
        setSize(300,200);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
}
```

```

public void addComponents() {
    JPanel p=new JPanel(); p.setLayout(new GridLayout(4,1,15,15));
    p.add(ajouterButton);
    p.add(supprimerButton);
    p.add(effacerButton);
    p.add(quitteButton);
    this.setLayout(new GridLayout(1,2));
    ajouterButton.addActionListener(this);
    supprimerButton.addActionListener(this);
    effacerButton.addActionListener(this);
    quitteButton.addActionListener(this);
    this.add(paysJListe);
    this.add(p);
    paysJListe.setModel(paysListModel);
    paysJListe.setBorder(new TitledBorder(null, "Liste des pays"));
}

public void ajouterListener() {
    String pays=JOptionPane.showInputDialog(this,"Saisir le nom du pays à ajouter");
    if(chercher(pays)==false)
        paysListModel.addElement(pays);
    else
        JOptionPane.showMessageDialog(this, "Le pays existe déjà", "Alerte",
        JOptionPane.WARNING_MESSAGE);
}

```

```

public boolean chercher(String pays) {
    boolean trouve=false;
    for(int i=0;i<paysListModel.size();i++)
        if(paysListModel.get(i).equalsIgnoreCase(pays)) {
            trouve=true;
            break;
        }
    return trouve;
}

public void supprimerListener() {
    int pays =paysJListe.getSelectedIndex();
    if(pays!=-1)
        paysListModel.remove(paysJListe.getSelectedIndex());
    else
        JOptionPane.showMessageDialog(this, "Veuillez sélectionner un pays dans la liste",
            "Information", JOptionPane.INFORMATION_MESSAGE);
}

public void effacerListener() {
    paysListModel.clear();
}

public void quitterListener() {
    this.dispose();
}

```

```
public void actionPerformed(ActionEvent e) {  
    if(e.getSource()==ajouterButton)  
        this.ajouterListener();  
    if(e.getSource()==supprimerButton)  
        this.supprimerListener();  
    if(e.getSource()==effacerButton)  
        this.effacerListener();  
    if(e.getSource()==quitterButton)  
        this.quitterListener();  
}  
}
```